University of Information Technology and Sciences

Computer Graphics and Multimedia Lab
CSE 358

Project Report
for
Triangular Jump

A fast-paced arcade-style game

Submitted By

Sadia Jannat
0432320005101108

Fatema Akter Nishi
043232005101121

Reyazul Islam
0432320005101146

Submitted To

Any Chowdhury
Lecturer
Department of CSE

Audity Ghosh
Lecturer
Department of CSE

1. Introduction

Triangular Jump is a high-performance 2D arcade side-scrolling game developed using C++ and modern OpenGL. The project serves as a comprehensive demonstration of computer graphics principles, including real-time rendering, procedural generation, particle systems, and physics-based animations.

The game challenges players to control a geometric character (a triangle) as it navigates through an infinite, procedurally generated obstacle course. The difficulty scales over time, requiring precise timing and reflexes.

2. Technical Specification

2.1 Core Technologies

- **Language**: C++11
- **Graphics API**: OpenGL 3.3 (Core Profile)
- **Windowing & Input**: GLFW 3
- **OpenGL Loader**: GLAD
- **Mathematics**: Custom Header-only Math Library (`math_utils.h`)
- **Build System**: Makefile (supports Linux and cross-compilation for Windows)

2.2 Custom Math Engine

To reduce dependency on external libraries like GLM and to demonstrate a deep understanding of linear algebra in graphics, the project utilizes a custom-built math library. Features include: - **Vector Math**: Implementation of `Vec2`, `Vec3`, and `Vec4` with operator overloading. - **Matrix Operations**: Column-major `Mat4` implementation for OpenGL compatibility. - **Transformations**: Optimized functions for Identity, Orthographic Projection, Translation, Rotation, and Scaling. - **Interpolation**: Linear interpolation (`lerp`) and clamping utilities for smooth animations.

3. Game Mechanics & Implementation

3.1 Player Dynamics

The player character is implemented in the `Player` class, featuring: - **Kinematics**: Velocity-based movement with gravity simulation. - **Jumping**: A impulse-based jump system with cooldowns. - **Squash & Stretch**: Procedural deformation (scaling) applied during jumps and landings to enhance the “feel” of the character. - **Dynamic Rotation**: The character rotates procedurally based on its airborne state and velocity.

3.2 Obstacle Management

The `ObstacleManager` handles the lifecycle of obstacles: - **Procedural Generation**: Randomly spawns obstacles from a pool of types: - `OBS_BLOCK`: Standard rectangular hazards. - `OBS_SPIKE`: Triangular spikes requiring tight jump timing. - `OBS_DOUBLE_BLOCK`: Layered hazards. - `OBS_LOW_BLOCK`: Hazards requiring specific movement patterns. - **Difficulty Scaling**: The scroll speed and spawn interval dynamically adjust as the player's score increases. - **Collision Detection**: Axis-Aligned Bounding Box (AABB) collision detection for efficient performance.

3.3 Particle System

A modular `ParticleSystem` is used to create visual flair: - **Trail Effects**: Particles emitted from the player's base during movement. - **Explosions**: High-density particle bursts triggered upon collision or death. - **Lifecycle Management**: Recycles particle objects to maintain high frame rates.

4. Graphics & Visual Effects

4.1 Rendering Pipeline

The `Renderer` class abstracts OpenGL calls, using a specialized shader-based pipeline: - **Instanced Drawing**: Batching geometry where possible. - **Shader Management**: Custom `Shader` class for loading, compiling, and managing GLSL programs. - **Alpha Blending**: Enabled for smooth particle transitions and UI overlays.

4.2 Visual Enhancements

- **Camera Shake**: A procedural screen-shake effect triggered by high-impact events (e.g., collisions).
- **Background Shaders**: A dedicated background shader providing a deep-space/neon aesthetic.
- **Animated Ground**: The ground texture scrolls and animates relative to the game speed to create the illusion of infinite movement.

4.3 HUD & User Interface

- **Real-time Scoring**: Tracks current distance and high scores.
- **State Machine**: Transitions smoothly between `GAME_MENU`, `GAME_ACTIVE`, and `GAME_OVER` states.
- **Performance Profiling**: Internal FPS counter for monitoring rendering efficiency.

5. Project Architecture

The codebase follows a modular, object-oriented design: - `main.cpp`: Entry point, window initialization, and main game loop. - `game.cpp`: Central hub for game logic, state management, and component coordination. - `renderer.cpp`: Low-level OpenGL abstraction and draw call management. - `player.cpp` / `obstacle.cpp`: Entity-specific logic and specialized behavior. - `resource_manager.cpp`: (Optional) Centralized loading of shaders and textures.

6. How to Build and Run

Prerequisites

- GCC/G++ (Linux) or MinGW (Windows)
- GLFW Development Libraries
- OpenGL Drivers

Build Commands

To build the project on Linux:

```
make linux
```

To run the game:

```
make run
```

7. Conclusion

Triangular Jump successfully demonstrates the integration of modern OpenGL techniques with custom game engine architecture. By eschewing high-level engines in favor of low-level API control, the project achieves high performance and a polished visual style while maintaining a clean, maintainable C++ codebase.